

植物大战僵尸 · C++ 植物编辑元学习项目

肖韵航 曹三省

2026 年 6 月

1 程序功能介绍

植物大战僵尸杂交版以全新的杂交植物风靡一时，但杂交植物只能由发行方提供。我们希望设计一款允许玩家自行编辑植物的游戏。结合本学期对 C++ 面向对象编程的学习，杂交植物恰好是一个非常契合的“对象”概念，因此我们将项目设计为一个元学习项目：引导玩家在游玩中学习 C++ 的面向对象编程方法。

游戏内提供 C++ 代码编辑器，可调用系统 g++/clang++ 编译代码并显示报错；玩家编译后可选择出战植物，像经典植物大战僵尸一样与僵尸战斗。



(a) 选择出战植物



(b) 杂交版中的部分植物



(c) 与僵尸战斗

图 1: 游戏核心玩法: 编辑 → 选择 → 战斗

2 项目各模块与类设计

层	职责	技术
Python 游戏引擎	UI 渲染、定时器调度、僵尸管理、阳光系统、碰撞检测	PyQt6
C++ 植物行为	植物逻辑（射击、产阳光、爆炸、动画切换）	C++17 动态库
C++ 组件系统	预定义组件，可组合为杂交植物	C++17 header-only
桥接层	ctypes 加载.dylib/.so，函数指针回调	Python ctypes
植物编辑器	游戏内代码编辑、编译、贴图合成、预览	PyQt6 + subprocess

整体架构

2.1 重构旧有代码并创建接口

项目基于一份课程作业中复刻的 C++/Qt 版植物大战僵尸（原始代码位于 PlantsVSZombies_old/），其植物行为直接硬编码在游戏逻辑中。为实现植物编辑功能并进行 C++ 面向对象教学，我们将植物单独抽象为植物类，并把游戏引擎整体重写为 Python (PyQt6) + C++：C++ 只负责植物行为逻辑，Python 负责游戏引擎、UI 渲染和资源管理。主代码见 PlantsVSZombies_pyqt6/（下文路径若无说明均在此目录下）。

为实现分层架构，我们设计了一套 C++ API 接口（cpp_api/pvz_api.h）：

- PVZGameAPI 结构体：17 个函数指针（6 个只读查询、3 个布尔判断、8 个动作调用）加 1 个上下文指针 void* ctx，作为 C++ 植物向 Python 查询状态、发出动作的统一通道。
- GameAPI 类：C++ 侧对 PVZGameAPI* 的面向对象封装，提供 shoot_pea()、raise_sun()、explode() 等方法，底层解引用函数指针。植物类只需调用这些方法即可实现行为逻辑。
- PVZ_REGISTER_PLANT 宏：自动生成 10 个 extern "C" 导出函数（create_plant、update_plant、get_plant_hp 等），供 Python 通过 ctypes 加载后直接调用。

Python 侧桥接层（plants/cpp_bridge.py）：通过 ctypes.CDLL 加载编译产物 .dylib/.so，封装为 PlantPlugin；PlantInstance 持有 opaque C++ 句柄，每 tick 调用 plugin.update(handle, api_struct)。GameAPIAdapter 将 GameScene 方法包装为 CFUNCTYPE 回调填入结构体传给 C++。该设计使 C++ 植物代码无需感知 PyQt、无需操作 UI 对象，实现了行为逻辑与表现层的解耦。

2.2 C++ 植物编辑系统

编辑系统的灵感来自“组合优于继承”思想，提供两层抽象：底层是零件（Component），代表植物能力的原子单位（如豌豆头、向日葵头）；上层是植物（HybridPlant），将零件自由组合为完整植物。两层分别对应编辑器的两种玩法模式。

2.1 零件系统（cpp_core/components.h）

Component 抽象基类：每个组件含三种属性 cost_（阳光花费）、cooldown_（冷却毫秒）、hp_（生命值贡献），以及纯虚函数 step(GameAPI&, HybridPlant*)，在其中调用 API 执行具体功能（检测僵尸、发射豌豆等），每一 tick 被调用一次。

```
class Component {
    int cost_, cooldown_, hp_;
public:
    Component(int c, int co, int hp) : cost_(c), cooldown_(co), hp_(hp) {}
    virtual ~Component() = default;
    virtual void step(GameAPI& api, HybridPlant* owner) = 0; // 纯虚：每 tick 执行
    int get_cost() const { return cost_; }
    int get_cooldown() const { return cooldown_; }
    int get_hp() const { return hp_; }
};
```

零件子类继承 Component，以 PeaHead 为例：

```
class PeaHead : public Component {
    int shot_number, time_prd;
public:
    PeaHead(int n, int prd)
        : Component(static_cast<int>(std::ceil(20000.0 / prd * n)), 7500, 300),
          shot_number(n), time_prd(prd) {}
    void step(GameAPI& api, HybridPlant* owner) override {
        if (!owner) return;
        if (owner->timer % time_prd == 0 && api.if_Zombies_ahead(owner->row, owner->col))
            api.shoot_pea(owner->row, owner->col, shot_number);
    }
};
```

};

MVP 阶段共实现四个零件子类，各有明确功能与成本公式：

组件	构造参数	行为	花费公式
PeaHead(n,prd)	n= 发射数, prd= 周期	每 prd tick 若前方有僵尸则射 n 发豌豆	$\text{ceil}(20000/\text{prd} \times n)$
SunHead(n,prd)	n= 产量, prd= 周期	每 prd tick 产 n 个阳光	$\text{ceil}(50000/\text{prd} \times n)$
WallNutHead()	无	提供 4000 HP，按血量分三级切换受损动画	50
PotatoMineHead()	无	1500 tick 后成熟，僵尸触碰时爆炸	25

组件内部维护自己的状态，用户编写植物构造函数时只需利用这些组件、传入参数，无需了解内部实现。例如 PeaHead(2, 200) 表示每 2 秒发射 2 发豌豆的双发射手行为。

2.2 植物系统

Plant 植物基类 (cpp_api/pvz_api.h)：所有 C++ 植物的基类，属性含位置与血量 row、col、hp，纯虚函数 Update(GameAPI&)=0 用于写每一步具体行为。**HybridPlant** 杂交植物类 (cpp_core/hybrid_plant.h)：public 继承 Plant，新增 timer 计数器、max_hp 与组件容器 std::vector<std::unique_ptr<Component>> components。

用户编写植物时遵循模板（游戏内置示例）：

```
#include "components.h"
class MyHybridPlantB : public HybridPlant {
public:
    MyHybridPlantB(int r, int c) : HybridPlant(r, c) {
        components.push_back(std::make_unique<SunHead>(1, 100)); // 添加组件
        components.push_back(std::make_unique<PeaHead>(1, 200)); // 参数含义见 components.h
        components.push_back(std::make_unique<WallNutHead>());
        finalize();
    }
};
// 类名(同一 cpp 内不可重名)，ID(唯一标识)，显示卡片名；贴图自动生成 custom_<ID>.png
PVZ_REGISTER_HYBRID(MyHybridPlantB, 400, "M300")
```

1. public 继承 HybridPlant；
2. 构造函数中用 components.push_back(std::make_unique<XXXHead>(...)) 添加组件——用户需读懂零件类原理并手动处理接口交互，从而练习面向对象编程；
3. 调用 finalize() 自动汇总 HP（各组件之和）、总花费（之和）、总冷却（最大值）；
4. 末尾用 PVZ_REGISTER_HYBRID 宏注册元数据（类名、ID、显示名）。

两种玩法模式：模式 A（组合官方组件）只能使用上述四个预定义零件、在 my_plant.cpp 中自由组合参数，components.h 只读；模式 B（编辑组件并组合）可同时编辑 components.h 与 my_plant.cpp，在理解 step() 接口前提下自定义新零件再组合，面向高级用户。

2.3 内置植物

项目含六个传统方式编写的内置植物 (cpp_plants/)：SunFlower、Peashooter、WallNut、PotatoMine、Repeater、DoubleSunShooter，直接继承 Plant 并用 PVZ_REGISTER_PLANT 注册。另有四个预编译杂交植物：PeaNut（豌豆 + 坚果）、SunNut（阳光 + 坚果）、MinePea（豌豆 + 土豆雷）、SunFortress（坚果 + 豌豆 + 阳光），继承 HybridPlant 随项目编译发布，作为学习范例与开箱即用内容。



(a) 模式 A: 组合官方组件 (components.h 只读)



(b) 可实现的杂交植物效果

图 2: 游戏内 C++ 植物编辑器与杂交效果

2.3 动态编译与数据储存

C++ 代码编译器

玩家提交编译后, 先用正则 `make_unique<(\w+)\>` 确认至少有一个组件 (否则报错), 再调用 `create_or_update_record()` 生成 cpp 文件, 同时调用 `plant_image()` 合成植物图片: 不超过三个组件时每个组件由”植物头 + 茎”构成 (官方提供头部图片, 模式 B 可自带); 超过三个时按纵向堆叠形成”汉堡”结构。最后在 Python 中调用终端按常规流程编译, 输出编译信息到窗口, 生成动态库 (mac 为 .dylib, Windows 为 .dll)。

```
compiler = shutil.which("clang++") or shutil.which("g++") or shutil.which("c++")
cmd = [compiler, "-std=c++17", "-shared", "-fPIC",
       "-I", str(base_dir / "cpp_api"), # 找 pvz_api.h
       "-I", str(base_dir / "cpp_core"), # 找 hybrid_plant.h / components.h
       "-I", str(saved_record.folder), # 找该植物自己的 components.h
       str(saved_record.plant_source_path), "-o", str(out_path)] # 输出 .dylib
result = subprocess.run(cmd, capture_output=True, text=True)
if result.returncode != 0:
    self.status_output.setPlainText(f"编译失败: \n{result.stderr}")
```



(a) ≤ 3 组件: 三头 + 茎布局



(b) > 3 组件: 纵向堆叠”汉堡”布局

图 3: 植物贴图自动合成

编译产物须被 Python 正确调用, 因此编译时已固定 API 接口名称, ctypes 即可找到对应函数 (由 PVZ_REGISTER_PLANT 宏生成的 `create_plant/destroy_plant/update_plant/get_plant_hp` 等 extern "C" 导出)。

基于 API 的 C++ 与 Python 交互

API 分为两类。游戏每隔 10ms 触发 `game_tick()` → `update_plants()`, C++ 执行 Update 时调用 Python 提供的 API: 例如 `api.shoot_pea(...)` 调用 `scene.api_shoot_pea(row,col,n)` 实际添加豌豆动画。即 C++ 只做决策 (何时发射、产阳光、爆炸), 所有画面/对象都由 Python 创建; C++ 既不持有 Qt 指针, 也不分配 Python 对象。

查询类 API (读状态, 有返回值)

方法	返回	语义
get_time()	int	当前游戏 tick 数
get_sun()	int	当前阳光总量
get_hp(r,c)	int	指定格植物血量
get_zombie_count(r)	int	整行存活僵尸数
count_zombies_ahead(r,c)	int	前方存活僵尸数
nearest_zombie_x(r,c)	int	最近僵尸 x, 无则 -1
if_Zombies_ahead(r,c)	bool	前方是否有僵尸
if_Zombies_touch(r,c)	bool	是否贴近 (引爆判定)
is_cell_empty(r,c)	bool	该格是否为空

动作类 API (执行动作, 无返回值)

方法	语义
shoot_pea(r,c,n=1)	发射 n 颗普通豌豆
shoot_snow_pea(r,c,n=1)	发射 n 颗寒冰豌豆
explode(r,c)	爆炸
explode_area(r,c,rad,dmg)	范围爆炸并清除自身
raise_sun(r,c)	产出一个阳光
change_animation(r,c,name)	替换该格贴图
set_hp(r,c,hp)	直接设置血量
damage_self(r,c,amt)	自损 amt 点血量

```
Source code --subprocess--> clang++/g++ --> plugins/plants/xxx.dylib
|
|               ctypes.CDLL (dlopen)
|
Python GameScene --update_plant(handle,&api)--> C++ plant->Update(api)
~
|
+----- api.shoot_pea() ----->
```

2.4 植物图鉴仓库 (plant_library.py)

图鉴是管理所有自定义植物的集中界面, 以分页卡片墙展示 (每页 8 个, 2×4)。每个卡片含缩略图、模式标签 (A/B)、中文名与 ID。功能围绕“浏览—筛选—管理”:

1. 勾选启用: 每个卡片配“启用”选框, 勾选的植物才出现在种子栏; 内置植物 (ID 1-5) 始终出场、不可取消。选中状态以 `set[int]` 储存至 `user_plants/selected_plants.json`。
2. 重新编辑/删除: 点击缩略图弹出菜单。“重新编辑”跳转编辑器; “删除”通过 `cleanup_orphan_custom_assets()` 清理残留的编译产物与图片。

数据来源 (`custom_plant_store.py`): 从 `user_plants/library/` 读取所有 `PlantRecord`, 每条记录是一个独立文件夹, 含 `metadata.json` (类名、ID、显示名、模式)、`plant.cpp` 与 `components.h`。该设计避免数据库依赖, 方便版本控制与手动维护; 预编译杂交植物则由虚拟条目 `_HybridEntry` 扫描 `plugins/plants/` 中不属于用户记录的 `.dylib` 得到。

2.5 植物演武场 (游戏模式, game_scene.py)

演武场即游戏模式, 植物被实例化并与僵尸大战。代码改造自原复刻代码, 但所有植物行为通过 API 接入游戏循环。

游戏循环: 以 `monitor_timer` (QTimer 间隔 10ms) 驱动, 每 tick 执行 `game_tick()`, 分两步: (1) `update_plants()` 遍历网格 `PlantInstance`, 从 C++ 拉取血量同步到 `self.plt_hp`, 调用 `plugin.update(...)` 执行 C++ 逻辑后再同步血量, `HP≤0` 则移除; (2) `monitor_zombies()` 处理僵尸移动、啃食、死亡动画与胜负判定。

阳光:`self.sun_num` 管理总量, `Seed.checksun()` 控制卡片遮罩显隐。向日葵组件经 `api.raise_sun()` 创建 `MySun`, 以 `QPropertyAnimation` 下落, 点击收集触发 `sun_collected` 信号, 10 秒未收集自动消失。

僵尸系统 (`zombies.py`): `make_zombie()` 创建四种僵尸, 内部含行走/啃食/死亡三状态, 经 `QMovie` 切换 GIF, 锥形与铁桶僵尸在 HP 降到阈值时切换受损外观。僵尸每 8 秒一波、共 50 波, 行列与类型由预设序列控制。

豌豆命中: `api_shoot_pea()` 创建豌豆 `QLabel` 并启动 `QTimer`, 每 33ms 右移 10 像素, 每帧 `hit()`



图 4：植物图鉴仓库：分页卡片墙与启用选框

检测碰撞，命中则 `get_hurt()` 减血并移除；多颗子弹用 `stagger QTimer` 实现 300ms 间隔。爆炸由 `api.explode()/explode_area()` 显示炸弹 GIF 并对范围僵尸造成伤害。

种子系统 (`seed.py`)：每个卡片为 `Seed(QPushButton)`，点击进入种植模式，光标变为植物缩略图；冷却由 `QPropertyAnimation` 遮罩下滑与 `QTimer.singleShot` 双重计时，冷却期或阳光不足时被灰色遮罩覆盖。

游戏启动：`CppPlantLoader.load_all()` 扫描 `plugins/plants/` 加载所有 `.dylib/.so`，按 `plant_id` 去重（同 ID 取最新 `mtime`），与图鉴勾选 ID 取交集构建种子栏，再播放倒计时动画后启动循环。

2.6 PyQt6 界面实现

UI 层全面基于 PyQt6，固定 960×720 分辨率，与原始 PvZ 一致。

场景管理：`MainWindow (main_window.py)` 继承 `QMainWindow`，用 `QStackedWidget` 管理场景切换。四个场景——主菜单、游戏、植物编辑器、植物图鉴——经 Qt 信号-槽串联；编辑器每次新建实例（状态独立），图鉴惰性地单例化（复用）。主菜单 (`menu.py`)：`bg.png` 全幅背景上定位 `ImageButton`（自定义 `QPushButton`，支持正常/悬停双态贴图、不规则遮罩边缘检测、按压位移）；弹窗由 `ImageDialog`（无边框 `QWidget`+ 贴图）实现。

核心自定义控件：`Seed`（种植态切换、阳光遮罩、冷却动画）、`MySun`（`Sun.gif` 动画、点击收集、超时消失）、`Zombie`（继承 `QLabel`，由 `ZombieSpec` 数据类驱动 `walk/eat/die`）、`CppHighlighter`（`QSyntaxHighlighter` 子类，用 `QRegularExpression` 实现 40+ 条语法规则）。辅助模块：`ui_helpers.py`（`asset()` 路径、`tinted_pixmap()` 半透明、`ImageDialog`）、`image_composer.py`（`QPainter.drawPixmap()` 逐层叠加合成，支持三头锚点与纵向堆叠）、`syntax_highlighter.py`（实时语法着色）。

3 项目小组分工

- 肖韵航：项目创意与架构设计，零件设计与植物选择菜单，报告对应部分。
- 曹三省：项目创意与架构设计，植物设计与编译器，报告对应部分。
- CodeX & Claude Code：完成原 C++ 代码转 Python 与 Python 侧 C++ API 模块。豆包 & Gemini：提供展示界面 UI 与三叉茎绘制。

4 项目总结与反思

本项目实现了基于植物大战僵尸杂交植物的 C++ 元学习功能，包括可扩展的 C++ 植物插件系统、内嵌于游戏的 C++ IDE 以及 Python-C++ 双层架构。

组件化的”零件 → 植物”两层抽象与教学目标高度契合：模式 A 让学生聚焦于组合（在已有零件中选择搭配），模式 B 开放零件自定义（需理解虚函数与接口），形成渐进式学习路径。双语言分工也是核心设计选择：将 C++ 职责严格限定在”植物行为逻辑”，使其脱离 PyQt 依赖，极大简化了玩家编写植物代码的负担——他们只需关注 `Update(GameAPI& api)` 一个函数，通过 `api.shoot_pea()`、`api.raise_sun()` 等语义明确的方法与游戏世界交互，无需了解 UI 渲染、僵尸 AI 或碰撞检测的实现细节。